SimpleTSD Multi-Draw Training: Memory-Safe Gradient Accumulation

Abstract

This note compares the SimpleTSD multi-draw training loop *before* and *after* the out-of-memory (OOM) fix. Multi-draw training averages the per-example loss over $M =$ `loss_mc` independent partial-mask draws to lower the pathwise gradient variance. The buggy version accumulated the loss of all M draws into one scalar and backpropagated *once*, so all M student computation graphs stayed alive simultaneously and activation memory scaled as $O(M)$, triggering OOM under DeepSpeed ZeRO-3. The fixed version backpropagates *each draw immediately* and accumulates gradients in DeepSpeed's buffer, so only one graph is alive at a time and activation memory is $O(1)$ in M. We show the two versions realise the *same* optimizer update (Proposition 1); only the peak memory and the number of backward calls differ.

1 Shared objective

Both versions optimise the identical objective. Fix one minibatch example $(x, y, \widehat{\mathcal{A}})$ with a nonempty selected forget set $\widehat{\mathcal{A}}$. For a single mask draw $\mathcal{M}$ with masked forget positions $F = \mathcal{M} \cap \widehat{\mathcal{A}}$ and self-distillation positions $R = \mathcal{M} \setminus \widehat{\mathcal{A}}$, the per-draw per-example loss is

$$\mathcal{L}_{\mathrm{ex}}(\mathcal{M}) = \begin{cases} \alpha\, \mathcal{L}_{\mathrm{f}}(\mathcal{M}) + (1-\alpha)\, \mathcal{L}_{\mathrm{d}}(\mathcal{M}), & R \neq \varnothing, \\ \mathcal{L}_{\mathrm{f}}(\mathcal{M}), & R = \varnothing, \end{cases} \tag{1}$$

where $\mathcal{L}_{\mathrm{f}}(\mathcal{M}) = \frac{1}{|F|} \sum_{i \in F} D_{\mathrm{KL}}(q_{\mathcal{M},i} \| p^{\theta}_{\mathcal{M},i})$ is the demotion term (gold token removed from the teacher and renormalised) and $\mathcal{L}_{\mathrm{d}}(\mathcal{M}) = \frac{1}{|R|} \sum_{i \in R} D_{\mathrm{KL}}(p^{0}_{\mathcal{M},i} \| p^{\theta}_{\mathcal{M},i})$ is the self-distillation term. The multi-draw example loss averages M independent draws $\mathcal{M}^{(1)}, \ldots, \mathcal{M}^{(M)}$:

$$\mathcal{L}(x, y, \widehat{\mathcal{A}}) \;=\; \frac{1}{M} \sum_{m=1}^{M} \mathcal{L}_{\mathrm{ex}}\big(\mathcal{M}^{(m)}\big). \tag{2}$$

The selected set $\widehat{\mathcal{A}}$ (PCSS, run once per example) and, in `cover` mode, the joint mask plan $\{\mathcal{M}^{(m)}\}$ are *mask-independent context*: computed once and shared by every draw. Only the student and teacher forward passes are repeated per draw.

2 Version A (before): accumulate then backpropagate once

Because ℓ references the output of every student forward, autograd cannot free any draw's activations until the single `backward` at the end. Peak activation memory is therefore

$$\mathrm{Mem}_{\mathrm{A}} \;=\; O(M) \times (\text{one student forward}), \tag{3}$$

which OOMs under ZeRO-3 for $M > 1$ once the sharded activations, offload buffers, and the teacher forward are added.

Algorithm 1 Buggy multi-draw step — one backward over M live graphs

Require: Example context (inputs, $\widehat{\mathcal{A}}$, cover plan); draws M.

$\ell \leftarrow 0$ ▷ scalar accumulator holding all M graphs
for $m = 1, \ldots, M$ **do**
  Draw $\mathcal{M}^{(m)}$; build $y_{\mathcal{M}^{(m)}}$.
  $o^{(m)} \leftarrow \text{student}(y_{\mathcal{M}^{(m)}})$ ▷ graph m kept alive
  $p^0 \leftarrow \text{teacher}(y_{\mathcal{M}^{(m)}})$ ▷ under `no_grad`
  $\ell \leftarrow \ell + \mathcal{L}_{\text{ex}}(\mathcal{M}^{(m)})$ ▷ graphs 1..m still alive
end for
$\ell \leftarrow \ell / M$
`accelerator.backward`$(\gamma\, \ell)$ ▷ one backward, M graphs

Algorithm 2 Fixed multi-draw step — one backward *per draw*

Require: Example context; draws M; HF boundary flag SYNC = `accelerator.sync_gradients`.

$s_0 \leftarrow$ SYNC ▷ save HF's boundary for this micro-step
for $m = 1, \ldots, M$ **do**
  set_sync_gradients$\big(s_0 \wedge (m = M)\big)$ ▷ boundary only on the last draw
  Draw $\mathcal{M}^{(m)}$; build $y_{\mathcal{M}^{(m)}}$.
  $o^{(m)} \leftarrow \text{student}(y_{\mathcal{M}^{(m)}})$
  $p^0 \leftarrow \text{teacher}(y_{\mathcal{M}^{(m)}})$ ▷ under `no_grad`
  $\ell_m \leftarrow \gamma\, \mathcal{L}_{\text{ex}}(\mathcal{M}^{(m)}) \,/\, M$
  `accelerator.backward`(ℓ_m) ▷ frees graph m; grads sum
end for
set_sync_gradients(s_0) ▷ restore HF's view for the outer loop

3 Version B (after): per-draw backward, gradient accumulation

Each `backward` frees draw m's graph before draw $m{+}1$ runs, so

$$\text{Mem}_{\text{B}} \;=\; O(1) \times (\text{one student forward}), \qquad \text{independent of } M. \tag{4}$$

Why the boundary flag matters (DeepSpeed ZeRO-3). Under `accelerate 1.11.0`, `accelerator.backward` forwards `sync_gradients` into the DeepSpeed engine wrapper's `backward`, which

1. sets DeepSpeed's gradient-accumulation boundary explicitly (via `set_gradient_accumulation_boundary`), overriding its internal micro-step counter; and
2. calls `engine.step()` (reduce-scatter, clip, optimizer step, zero-grad, LR step) *only* when `sync_gradients` is true.

Forcing the flag false on draws 1..$M{-}1$ accumulates their gradients into DeepSpeed's buffer with *no* reduce/step; the last draw performs the reduce and the single optimizer step — but only on HF's own accumulation boundary (s_0 true). On non-boundary micro-steps every draw stays false and merely accumulates, exactly as ordinary gradient accumulation across micro-batches. The outer HF `optimizer.step()` is a no-op under DeepSpeed, so there is no double step.

4 Equivalence of the two versions

Proposition 1 (Identical optimizer update)**.** *For fixed draws $\{\mathcal{M}^{(m)}\}$, Versions A and B produce the same accumulated parameter gradient, hence the same optimizer step.*

Proof. The teacher forward is under `no_grad`, so $q_{\mathcal{M},i}$ and $p^0_{\mathcal{M},i}$ are constants w.r.t. θ; the only θ-dependence is through $p^\theta_{\mathcal{M}^{(m)}}$, and distinct draws share no activation. Version A backpropagates $\gamma \frac{1}{M} \sum_m \mathcal{L}_{\text{ex}}(\mathcal{M}^{(m)})$ once, Version B backpropagates $\ell_m = \gamma\, \mathcal{L}_{\text{ex}}(\mathcal{M}^{(m)})/M$ separately. PyTorch/DeepSpeed *accumulate* (sum)

gradients across `backward` calls with no intervening reduce or zero-grad (boundary false on $m < M$), so

$$g_B \;=\; \sum_{m=1}^{M} \nabla_\theta \ell_m \;=\; \frac{\gamma}{M} \sum_{m=1}^{M} \nabla_\theta \mathcal{L}_{\text{ex}}(\mathcal{M}^{(m)}) \;=\; g_A.$$

Because $g_A = g_B$ and the reduce-scatter is linear, both trigger one identical `engine.step()` on the boundary. The explicit $1/M$ scaling is orthogonal to DeepSpeed's own $1/$`gradient_accumulation_steps` scaling (accelerate does not divide the loss for DeepSpeed), so relative scale versus the single-draw path is preserved. □

Remark 1 (Not bit-identical). Versions A and B are mathematically equal but need not be bit-identical: summation order in gradient accumulation and the timing of RNG draws differ. For $M = 1$ the fixed code takes the original single-draw path (`super().training_step`) verbatim, so that case is unchanged.

5 Summary

Table 1: Version A (before) vs. Version B (after) for $M =$ `loss_mc` draws.

Property	**A: before (buggy)**	**B: after (fixed)**
Objective (Eq. 2)	mean over M draws	same
Optimizer update	identical (Prop. 1)	
`backward` calls / step	1	M
Live student graphs (peak)	M	1
Activation memory	$O(M)$	$O(1)$
DeepSpeed `engine.step()`	once (last)	once (last, boundary only)
$M = 1$ behaviour	original single-draw path (unchanged)	
Result under ZeRO-3, $M > 1$	OOM	trains

The fix changes *how* the gradient is materialised, not *what* is optimised: the same mean-over-draws objective and the same optimizer step, at $O(1)$ instead of $O(M)$ activation memory.